



Leveraging textual properties of bug reports to localize relevant source files



Reza Gharibi*, Amir Hossein Rasekh, Mohammad Hadi Sadreddini,
Seyed Mostafa Fakhrahmad*

Department of Computer Science & Engineering & IT, School of Electrical and Computer Engineering, Shiraz University, Shiraz, Iran

ARTICLE INFO

Keywords:

Bug localization
Bug report
Classification
Information retrieval
Textual analysis

ABSTRACT

Bug reports are an essential part of a software project's life cycle since resolving them improves the project's quality. When a new bug report is received, developers usually need to reproduce the bug and perform code review to locate the bug and assign it to be fixed. However, the huge number of bug reports and the increasing size of software projects make this process tedious and time-consuming. To solve this issue, bug localization techniques try to rank all the source files of a project with respect to how likely they are to contain a bug. This process reduces the search space of source files and helps developers to find relevant source files quicker. In this paper, we propose a multi-component bug localization approach that leverages different textual properties of bug reports and source files as well as the relations between previously fixed bug reports and a newly received one. Our approach uses information retrieval, textual matching, stack trace analysis, and multi-label classification to improve the performance of bug localization. We evaluate the performance of the proposed approach on three open source software projects (i.e., AspectJ, SWT, and ZXing) and the results show that it can rank appropriate source files for more than 52% of bugs by recommending only one source file and 78% by recommending ten files. It also improves the MRR and MAP values compared to several existing state-of-the-art bug localization approaches.

1. Introduction

A software bug is an error, flaw, failure or fault in a computer program that produces an incorrect or unexpected result. The quality of a software project is an important aspect for its success. Although different software quality control processes like software testing and software inspection are used, the increasing size of software and the limited development resources make developers release their software while still containing bugs. To deal with this problem, software projects set up bug tracking systems such as Bugzilla¹ and MantisBT² to keep track of bugs in their product effectively and then ask users all over the world to report bugs that they have confronted. However, this process can also create a new challenge for developers.

During the development and post-release life cycle of a software product, especially in a popular and well-known system, the number of bug reports can grow large and easily become tedious to handle. For example, there were 3,389 bug reports created for the

* Corresponding authors.

E-mail addresses: gharibi@cse.shirazu.ac.ir (R. Gharibi), ahrasekh@shirazu.ac.ir (A.H. Rasekh), sadredin@shirazu.ac.ir (M.H. Sadreddini), fakhrahmad@shirazu.ac.ir (S.M. Fakhrahmad).

¹ <https://www.bugzilla.org/>.

² <https://www.mantisbt.org/>.

<https://doi.org/10.1016/j.ipm.2018.07.004>

Received 9 February 2018; Received in revised form 14 July 2018; Accepted 15 July 2018

Available online 31 July 2018

0306-4573/ © 2018 Elsevier Ltd. All rights reserved.

Eclipse platform product in 2013 alone (Ye, Bunesco, & Liu, 2014) and Mozilla project receives on average 300 new bug reports every day (Shokripour, Anvik, Kasirun, & Zamani, 2013). These numbers show that it's essential for the development teams to efficiently locate buggy files and resolve the problem.

After a bug is reported and confirmed, the development team should find which source files contain the problem and who will be assigned to make the code changes necessary to fix the fault (Jeong, Kim, & Zimmermann, 2009). As the software projects also become bigger and more complicated, one of the most time-consuming and difficult tasks is finding the buggy files that are responsible for a reported bug among hundreds or thousands of source files (Anvik, Hiew, & Murphy, 2005). Furthermore, developers who have to maintain and work on unfamiliar projects need to put a lot of effort in comprehending and understanding different parts of the program to find higher level concepts that are described by the bug report. Therefore, it would be beneficial to have an automatic system help us in this process that recommend buggy source files to developers based on the given bug report.

Researchers have proposed approaches to automate the process of finding the faulty source files. These approaches rank source files according to their relevance to the given bug report and make the developers' search space smaller. Developers then examine the files from the beginning of the ranked list to find the relevant faulty files.

One category of these approaches is spectrum-based that need the spectra information between different executions of the program to help locate faults (Abreu, Zoetewij, Golsteijn, & van Gemund, 2009; Jones & Harrold, 2005; Le, Lo, Le Goues, & Grunske, 2016; Liu, Fei, Yan, Han, & Midkiff, 2006). Spectrum-based approaches usually need a test suite to execute the program and collect its execution information; while the lack of test cases in software projects makes it hard to collect many correct and failing execution traces. Another category of bug localization approaches is information retrieval (IR)-based techniques that are static and don't require runtime information of the program (Rahman, Ganguly, & Sakib, 2015; Saha, Lease, Khurshid, & Perry, 2013; Wang & Lo, 2014; Wong et al., 2014; Zhou, Tong, Chen, & Han, 2017; Zhou, Zhang, & Lo, 2012). These approaches locate the bug-relevant files based on a given bug report. In fact, the text in the summary and description of a given bug report is considered as a query and source files are documents that are ranked based on their relevance to the bug report's content.

As shown in Fig. 1, there are similar word tokens in different parts of a bug report³ and its associated source file. The intuition here is that software elements that share many common word tokens with the given bug report are likely to be relevant to the bug and can help locate the appropriate source file for each bug report quicker (Zhou et al., 2012). Apart from word tokens, full identifiers are often present in bug reports and can further help in going toward the right source file (Saha et al., 2013). Some matching positions in the source file are more important than others. We can give more weight to matching terms or word tokens in these positions in order to avoid their effect loss in finding the relevant source file. Words in bug reports also have different importance. For example, this can be seen in Fig. 2 as words with specific part-of-speech or POS tags are more important, and mostly nouns and verbs are the common words between the bug report and source code.

Some bug reports contain a stack trace and the relevant file name for a bug report can be found there. It has been shown that the relevant source file of about 90% of bugs is located within the top 10 stack frames (Schröter, Bettenburg, & Premraj, 2010), and as expected, most of the times it is in the first frame. Fig. 3 shows an example of a bug report⁴ containing a stack trace of a component that has crashed. The relevant source file for this bug report is "ProgramElement.java", which is located in the first frame of the stack trace.

Sometimes the terms used to describe the bug in the bug report are not the same as the ones used in the source code, but if we consider them semantically, they are close in meaning. For example in Fig. 4, the word "camera" is used, and the related source file to this bug report has the term "capture" in its name. We know that "camera" and "capture" are related to each other; therefore we can use semantic similarity to overcome this lexical mismatch between the bug report and source file. There are many different mismatches in what bug reports contain and what developers consider useful in them to better localize the bugs (Bettenburg et al., 2008), but lexical mismatch makes it harder for a bug localization system. In this case, some domain knowledge about the software project is needed to locate relevant files (Ye et al., 2014).

Previously fixed source files may be responsible for newer similar bugs (Murphy-Hill, Zimmermann, Bird, & Nagappan, 2013), and sometimes people submit bug reports similar to these already fixed bug reports that may affect the same source file. Since usually there are already fixed bug reports available in bug tracking systems of software projects, common tokens shared between new and fixed reports can be used to further improve the bug localization performance.

Although many bug localization approaches are proposed, there are still some gaps that can be covered to improve their accuracy. Motivated by these observations, in this paper, we propose a five-component bug localization approach that combines various textual aspects of bug reports and source files to locate relevant source files for each bug report. The components consist of a token matching component that tries to find exact matches of some specific textual tokens of bug reports in specific positions of source files; a Vector Space Model (VSM) similarity component that utilizes a variation of the revised Vector Space Model (rVSM) (Zhou et al., 2012) with specifically extracted word tokens to better score source files for a bug report; a stack trace component that uses stack traces in bug reports if available; a semantic similarity component, and finally a fixed bug reports component that uses previously fixed bug reports and a multi-label classification algorithm to score suspicious faulty source files according to existing solved bug reports.

As the main contribution of this paper, we first develop these five components to get separate ranking scores for each source file. Then we combine these scores to get a final ranking score and use that to rank relevant source files with respect to each bug report. After that, we try to show how good our approach works on the bug reports of real-world software projects. Therefore, we evaluate

³ https://bugs.eclipse.org/bugs/show_bug.cgi?id=80720.

⁴ https://bugs.eclipse.org/bugs/show_bug.cgi?id=43709.

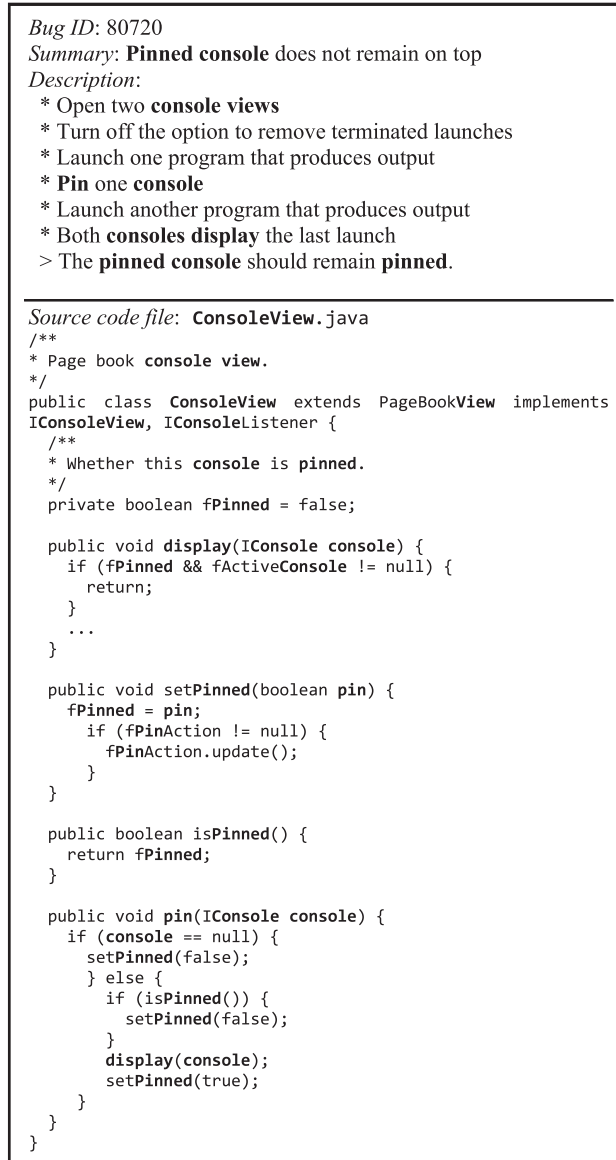


Fig. 1. An example of similar word tokens in a bug report and its relevant source file. Common terms are shown in bold.

Pinned	console	does not remain on top
VERB	NOUN	
Open two	console	views
	NOUN	NOUN
Pin	one	console
VERB		NOUN
Both	consoles	display the last launch
	NOUN	VERB
The	pinned	console should remain pinned
	ADJ	NOUN VERB

Fig. 2. POS tags for the bug report's common terms.

our approach on three open source software projects, namely, AspectJ, SWT, and ZXing. We also compare our approach with several state-of-the-art bug localization approaches including BugLocator proposed by Zhou et al. (2012), BLUIR proposed by Saha et al. (2013), BRTracer proposed by Wong et al. (2014), AmaLgam proposed by Wang and Lo (2014), and Rahman et al. (2015) approach. Results show that on average for the benchmark projects, our approach can rank appropriate source files for 52.5% of bugs

Bug ID: 43709
Summary: structure view crash when extending library aspects
Description:
 ...
 Here is the stack trace in AJDT:
 java.lang.NullPointerException
 at org.aspectj.asm.internal.ProgramElement.toLinkLabelString(ProgramElement.java:403)
 at org.eclipse.ajdt.internal.core.AJDTStructureViewNode.getLabel(AJDTStructureViewNode.java:171)
 at org.eclipse.ajdt.internal.core.AJDTStructureViewNodeAdapter.getLabel(AJDTStructureViewNodeAdapter.java:89)
 at org.eclipse.ui.model.WorkbenchLabelProvider.getText(WorkbenchLabelProvider.java:142)
 at org.eclipse.jface.viewers.TreeViewer.doUpdateItem(TreeViewer.java:95)
 at org.eclipse.jface.viewers.AbstractTreeViewer\$UpdateItemSafeRunnable.run(AbstractTreeViewer.java:87)
 at org.eclipse.core.internal.runtime.InternalPlatform.run(InternalPlatform.java:1006)
 at org.eclipse.core.runtime.Platform.run(Platform.java:413)
 ...

Source code file: **ProgramElement.java**

Fig. 3. An example of a bug report containing a stack trace.

Bug ID: 376
Description:
 ...
 Version 48 for android I've run tests with the **camera**, and it appears it does not let you set an initial zoom.
 ...

Source code files:
CaptureActivity.java
CaptureActivityHandler.java
 ...

Fig. 4. Lexical mismatch between bug report terms and source file names.

in top 1 and 78.2% in top 10 and also improves the mean reciprocal rank (MRR) and mean average precision (MAP) values. Considering the MAP value, the improvement is 37.8%, 25.4%, 16.8%, and 12.5% over BugLocator, BLUIR, BRTracer, and AmaLgam, respectively. In the same order, the MRR value has the improvement of 38.2%, 25.9%, 17.8%, and 13.1%.

This paper significantly extends our preliminary work (Gharibi, Rasekh, & Sadreddini, 2017). Two new components (i.e., stack trace and semantic similarity components) are added, and previous components are further improved. More extensive evaluation is also done. We added one more software project to our benchmark dataset and compared our results to four more previous studies with and without the use of previously fixed bug reports. In addition, the rest of the paper is also substantially expanded with new content.

The remainder of the paper is structured as follows. In Section 2, we review how this work relates to other research. The details of our proposed approach are described in Section 3. The experimental setup of the model is explained in Section 4. Section 5, presents the results and discusses our main findings. Finally, this is followed by Section 6, where we draw conclusions and describe future work.

2. Related work

In this section, we briefly review the related work. We divide related studies into four categories. First, textual and IR-based bug localization approaches are discussed. Then we describe some feature location works and spectrum-based methods. Finally, other related studies are also shown.

2.1. Textual and IR-based bug localization

Several approaches have been proposed for automatic bug localization using IR and textual analysis techniques. Lukins, Kraft, and Etkorn (2008) presented a latent Dirichlet allocation (LDA)-based method for bug localization. They applied their method to three case studies (Eclipse, Mozilla and, Rhino) with a small number of bug reports. They compared their results to a latent semantic indexing (LSI)-based method, and it showed that their LDA-based method works at least as well as the LSI techniques.

Nguyen, Nguyen, Al-Kofahi, Nguyen, and Nguyen (2011) proposed BugScout, a topic-based approach that tries to narrow down the search space of buggy source files. BugScout uses a specialized topic model based on LDA that represents technical aspects as

topics in the textual contents of bug reports and source files; it then correlates bug reports and corresponding buggy files via their shared topics. Their system has two components. The S-component is a model for source files, and the B-component is an extended LDA model for bug reports in which a bug report is modeled as a document that is influenced not only by its own topic distribution parameter but also by the topic distribution parameters of the buggy source files corresponding to that bug report. For ranking, they train their model on historical data to estimate the topic distribution of a new bug report and compare it with the topic distributions of all the source files. They also give more weight to the source files that have had more defects in the past. Their evaluation on several software projects including Jazz, Eclipse, AspectJ, and ArgoUML shows that BugScout can recommend buggy files correctly up to 45% of the cases with a recommended ranked list of 10 files.

Zhou et al. (2012) proposed BugLocator which uses the combined scores of a rVSM (rVSMscore) and similar previously fixed bug reports (SimiScore) to rank relevant source files for new bug reports. They first compute the similarity of a bug report to source files using their rVSM and give each source file a similarity score. Then for each source code file, based on the similarity of a given bug report to old bug reports, they calculate a score using VSM. Finally, they linearly combine the scores of their components using a single α parameter and rank source files based on this final score. BugLocator's effectiveness has been evaluated on the dataset of four open source projects that are also used by us to evaluate our approach. Their results showed that BugLocator outperforms previously existing bug localization methods. We adopt a variation of their revised Vector Space Model in our VSM similarity component and use more meaningful terms from bug reports and source files to improve its effectiveness. We also utilize the previously fixed bug reports in our approach to better localize faulty files for newer bug reports, however, we use a multi-label classification algorithm in our fixed bug reports component instead of using a simple VSM similarity.

Saha et al. (2013) argue that existing approaches treat source code files as one whole unit and bug reports as one plain text where important information like class names and method names often get lost in the relatively large number of terms. To overcome this issue, they presented BLUIR that employs the structured IR model and takes advantage of bug similarity data if available. They transform source code files to their constituent parts (i.e., a source code file is separated into four groups of identifiers: class names, method names, variable names, and comments) and bug reports are also divided into their summary and description. Then they use structured IR with the Okapi BM25 scoring to compute similarities between the eight combinations of source file parts and bug report parts and sum them into a final score to rank source files. They evaluated BLUIR on the same dataset of projects as BugLocator. Results showed that BLUIR matched or outperformed previously existing bug localization tools.

Moreno, Treadway, Marcus, and Shen (2014) proposed a technique called Lobster that uses stack traces found in bug reports and a dependency graph of the software as well as textual similarity to find relevant source files to a bug report. The dependency graph is used with the intuition that, while buggy files are not always present in the stack trace, they may be structurally related to the source files in the stack trace. Lobster combines its textual similarity score, which is given by the Lucene tool, and its structural similarity score, which is calculated from stack traces and the dependency graph, to rank source files. They evaluated the approach with only bug reports containing a stack trace and concluded that using stack traces improves the performance compared to only using the Lucene's VSM.

Wong et al. (2014) used segmentation and stack trace analysis to improve the performance of bug localization. Given a bug report, they divide source files into a series of equally sized segments and use the most similar segment to the bug report to represent each source file. They also analyzed bug reports and extracted stack traces to favor faulty source files in stack traces. The implementation of their tool is called BRTracer, and they built it on top of BugLocator. It is evaluated using the same dataset as BugLocator, and they achieved better performance compared to BugLocator. We got the intuition of using stack traces from this study, but the way we utilize them is different both in source file extraction and in source file scoring based on files that appear in the stack trace. Our source file extraction is more precise since it not only uses stack frames containing the exact source file names but also tries to extract source file names for stack frames in Unknown Source form. Additionally, the package name of source files is also checked (if exists) to avoid getting false positives from same file names in different packages. For the case of scoring, BRTracer gives a constant score to the source files with the rank more than 10 and those that are directly used within the stack trace source files.

The historical data of changes to source files that are stored in version control systems are an important information source to improve the effectiveness of a bug localization system. Sisman and Kak (2012) used version histories of source files in an IR-based bug localization system and showed how version histories of a software project could be used to estimate a prior probability for defect proneness associated with the files in a given version of the project. They then used this probability in an IR framework to find the cause of a bug. They have evaluated their system on NCSA Mosaic web browser. In another research (Sisman & Kak, 2013), they proposed a query reformulation framework for bug localization in a manner that requires no further input from the user. It tries to enrich the user's search query with certain additional terms close to the original query gotten from the highest ranked artifacts retrieved in response to the initial query. Their framework is evaluated on two software projects (Eclipse and Chrome web browser) with more than 4,000 queries that are constructed using the titles of bug reports.

There are also learning to rank algorithms for ranking all the source files of a project with respect to how likely they are to contain the cause of a bug. Ye et al. (2014) extracted six features and gave scores to source files by the weighted combination of these features. They tried to leverage domain knowledge by functional decomposition of source code files into methods, bug-fixing and code change history, and the API documentation of software projects. The API description of code components is used to cover the lexical gap between bug reports and source files. They evaluated their system on six large-scale open source Java projects and the results showed that their method is better than previous methods in finding relevant files. In our approach, we mainly concentrate on only using the textual properties of bug reports and source files, and we don't dig further into using other information that can be mined from software repositories and bug databases; however, we leave utilizing this data for our future work. The amount of data available in versioning systems, issue trackers, and many other repositories is continuously growing over time. Mining specific data

from these software repositories can give opportunities to improve bug localization (Bavota, 2016).

Wang and Lo (2014) proposed Amalgam that combines historical data, similar reports, and structural information to achieve higher accuracy for locating relevant buggy files. Amalgam uses BugLocator's SimiScore in its similar report component and BLUIR's structured retrieval in its structure component. The version history component scores files based on the number of bug-fixing commits that touch each of them in past 15 days. Their approach is evaluated the same way as BugLocator and is compared to Sisman and Kak (2012), BugLocator, and BLUIR. Their results were better or equal to previous approaches except for MAP in the ZXing project. The same authors in another study (Wang & Lo, 2016) added two more sources of information to their system to further improve the method. The new method is called Amalgam+, and it incorporates two new components, which are stack traces and reporter information, to previous components. The scores of these components are combined linearly with a contribution weight for each component. Weights are then estimated using a genetic algorithm. The results outperformed or matched previous methods, especially for projects that have more historical data.

Rahman et al. (2015) extended BugLocator and added frequently fixed files and class name matching to it. The frequently fixed files score is computed based on the number of occurrences of a fixed file in the previous bug reports, and the class name matching score is computed based on class names that appear in the bug reports. The final ranking score is computed based on the combination of these scores. The approach is evaluated with two of the projects from BugLocator dataset and a new project of their own. Wen, Wu, and Cheung (2016) discussed the coarse granularity of files and proposed Locus, which is an approach to locate bugs both at the file level and at a finer granularity of source file changes. Their idea is that bug-inducing changes in source files are an important clue for developers to find and fix a bug. To achieve their goal, they used changed lines of source files, unchanged lines, commit logs, commit timestamp and change history. Locus is evaluated both at source file level and at change level.

Dealing with lexical mismatches between terms of bug reports and source files is an important subject. Lam, Nguyen, Nguyen, and Nguyen (2015) used a deep neural network (DNN) to learn the relevance between the terms of bug reports, source files, and the documentation. They fed this relevance feature and a textual similarity feature computed using rVSM (Zhou et al., 2012) and a project's bug-fixing history to another DNN to combine them and give a final ranking score. Their new model is called HyLoc, and it is evaluated on several real-world projects. Results showed higher accuracy compared to some previous approaches. In addition to HyLoc, Uneno, Mizuno, and Choi (2016) proposed DrewBL to overcome the lexical mismatch problem. DrewBL gives terms from bug reports and source files to Word2Vec (Mikolov, Sutskever, Chen, Corrado, & Dean, 2013) and builds a semantic Vector Space Model (sVSM) to compute the relevance between them. DrewBL by itself doesn't have high accuracy, so they combined its score with a component score based on tokens from component names and the scores of two existing approaches BugLocator and BugSpots, which uses the bug-fixing history. The combined approach is called CombBL and it achieves better accuracy than DrewDL. In our work, we use pre-trained GloVe (Pennington, Socher, & Manning, 2014) word vectors to solve the lexical mismatch issue. Pre-trained word vectors are trained on a larger dataset and are a robust choice when the software project's historical data is not large enough, and the neighboring relation among source file terms are not like usual natural language texts to train good word vectors.

In a recent study, Zhou et al. (2017) presented a method that leverages the POS features of bug reports and the invocation relationship among source files. Their method has two modules that recommend source files for two different user demands. The first module tries to maximize the accuracy of the first recommended file, and the other one aims at improving the accuracy of the whole list of defected files. The method is evaluated on some open source projects, and they have reported separate results for their modules. They also compared their results to the version of BugLocator that doesn't use similar bug reports.

Table 1 shows a summary of techniques used by the approaches we compare to our approach. It should be noted that although some approaches use similar techniques, they are not utilizing them exactly the same.

There are also some studies that combined and compared different basic IR models for the task of bug localization. Rao and Kak (2011) compared five generic text models and certain composite variations of them for the case of bug localization. They evaluated their models on iBUGS dataset benchmark, and their results concluded that simple text models like unigram model or VSM are more effective than some complicated models like LDA or LSI. Wang, Lo, and Lawall (2014) combined various VSM variants with different weighting schemes and used a genetic algorithm to explore the space of their possible compositions and output a heuristically near-optimal composite model. They showed that this approach can achieve better performance than VSM with the standard tf-idf weighting scheme. Khatiwada, Tushev, and Mahmoud (2018) also compared some information-theoretic IR methods, including pointwise mutual information (PMI) and normalized Google distance (NGD) against classical IR methods that are commonly used in the bug localization research. Their results showed that information-theoretic IR methods perform better than classical IR methods,

Table 1
Comparison of the techniques used by state-of-the-art bug localization approaches.

Approach	POS tagging	Token matching	VSM similarity	Stack trace	Semantic Similarity	Fixed bug reports	Version history	Call graph
BugLocator (Zhou et al., 2012)			×			×		
BLUIR (Saha et al., 2013)			×			×		
BRTracer (Wong et al., 2014)			×	×		×		
Amalgam (Wang & Lo, 2014)			×			×	×	
Rahman et al. (2015)		×	×			×		
Zhou et al. (2017)	×		×					×
Our approach	×	×	×	×	×	×		

providing enough semantics necessary to overcome the vocabulary mismatch problem affecting methods such as VSM to establish relations between bug reports and code artifacts successfully.

2.2. Feature/concept location

Identifying where to make changes in a software system in response to a change request is known as feature or concept location (Rajlich & Wilde, 2002). Bug localization can be seen as an instance of feature location where the change request is given as a bug report, and developers need to change the code to fix the wrong behavior of the program. Many feature location works also applied their system to the task of bug localization. Marcus, Sergeyev, Rajlich, and Maletic (2004) presented an IR approach for concept location. They have used LSI to map concepts expressed in natural language by the programmer to the relevant parts of the source code. Poshyvanyk, Gueheneuc, Marcus, Antoniol, and Rajlich (2007) proposed a feature location technique called PROMESIR. They combined the results of two previously developed techniques for feature location that provide complementary results. The first technique is an IR method based on LSI and the second technique is a scenario-based probabilistic ranking of events observed while executing the program under given scenarios. They applied PROMESIR to locate eight bugs in Mozilla and Eclipse systems. In another study, Gay, Haiduc, Marcus, and Menzies (2009) combined the IR-based concept location with explicit relevance feedback. The goal of their method is to reduce developer effort in query reformulation. They evaluated their approach using seven Eclipse bug reports and patch description of jEdit and Adempiere projects. Eddy, Kraft, and Gray (2018) also showed how weighting terms from different locations in source code can improve an LDA-based feature location technique. There is also a more complete systematic literature survey of feature location techniques available by Dit, Revelle, Gethers, and Poshyvanyk (2013) which has a comprehensive review of 89 feature location papers.

2.3. Spectrum-based bug localization

A number of studies used spectrum-based bug localization techniques. These methods need runtime information and usually require a test suite and collect passing and failing execution information to locate bugs. Spectrum-based methods can be more accurate in exactly finding the buggy statement or basic blocks; they then rank these program elements by their fault suspiciousness. Tarantula (Jones & Harrold, 2005) is one of the popular spectrum-based techniques, and its basic idea is that a program element is considered to be more suspicious if it appears more frequently in failed executions than in correct ones.

Liu et al. (2006) proposed a statistical approach called SOBER to localize software faults without prior knowledge of program semantics. SOBER models the predicate evaluation in both correct and incorrect program executions and considers a predicate as fault-relevant if its evaluation pattern in incorrect executions significantly diverges from correct ones. They evaluated SOBER using Siemens suite and two case studies with grep and bc.

Le et al. (2016) recently presented Savant that uses likely invariant diffs and suspiciousness scores as features in a learning to rank strategy to rank methods based on their likelihood of being a cause of a failure. Savant has four steps: method clustering and test case selection, invariant mining, feature extraction, and method ranking. They have evaluated Savant on 357 real-life bugs from five programs of the Defects4J benchmark. Le, Oentaryo, and Lo (2015) also have combined IR-based and spectrum-based techniques and created an approach called adaptive multi-modal bug localization (AML) to cover the weak points of both methods and better localize new bugs.

2.4. Other related studies

In this subsection, other related studies that can be useful for the task of bug localization are described.

Bug prediction techniques and metrics are important and can help improve bug localization by showing fault-prone locations and source files (Kim, Zimmermann, Whitehead Jr., & Zeller, 2007; Zhang, 2009). The length score we incorporate in our VSM similarity component is one usage of bug prediction metrics. Automatic bug assignment to a development team or an individual developer is vital for better bug correction, while the manual bug assignment is labor-intensive, error-prone, and delays bug corrections (Jonsson et al., 2016). Moreover, it can help a localization tool to better narrow its results based on the part of the source code that the assigned developers wrote.

Sometimes two or more bug reports correspond to the same defect. Assuming different people describe a bug differently and in their own set of words, detecting and merging duplicate bug reports can give more information about a given bug and the opportunity to have more lexical resources to localize a bug. Several automatic approaches to detect duplicate bug reports have been proposed in the literature (Runeson, Alexandersson, & Nyholm, 2007; Sun, Lo, Wang, Jiang, & Khoo, 2010; Wang, Zhang, Xie, Anvik, & Sun, 2008). Bug report summarization (Jiang, Nazar, Zhang, Zhang, & Ren, 2017; Mani, Catherine, Sinha, & Dubey, 2012; Mehta & Majumder, 2018; Rastkar, Murphy, & Murray, 2014) can improve the bug localization process and help extract valuable information from resolved bug reports. POS tagging can help to find more important words in the bug report (Tian & Lo, 2015), as we also use POS tagging in different parts of our system to utilize more informative words.

Another related area of research to our work is software traceability that tries to link different software artifacts (such as source code, documentation, and tests) to each other (Borg, Runeson, & Ardö, 2014; Cleland-Huang, Gotel, Huffman Hayes, Mäder, & Zisman, 2014). Tools automating change impact analysis (CIA) use IR-based trace recovery techniques and mine software repositories to recommend artifacts impacted by a change in other development artifacts like the potential impact of resolving issue reports (Borg, Wnuk, Regnell, & Runeson, 2017). There are also studies regarding the practical usefulness of automatic bug localization tools

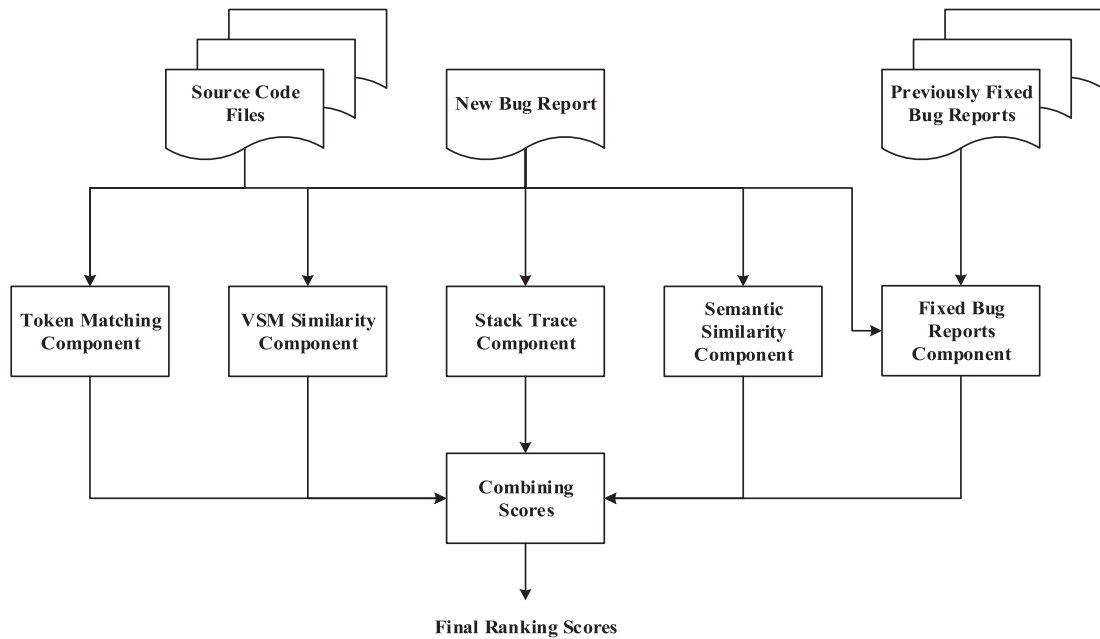


Fig. 5. The overall structure of our approach.

which show how developers use and benefit from automated bug localization tools (Le, Thung, & Lo, 2014; Parnin & Orso, 2011; Wang, Parnin, & Orso, 2015).

3. Proposed approach

In this section, we describe our proposed approach for building an automatic bug localization tool that locates buggy files based on the information it gathers from the given bug reports. Our approach consists of a parsing and preprocessing phase, and five components that are the token matching, VSM similarity, stack trace, semantic similarity, and fixed bug reports. Each of the mentioned components gives a score to source files with respect to each bug report. Then these scores are combined to get a final ranking score that is used to rank relevant source files. The overall structure of the approach and its components are shown in Fig. 5.

In the following subsections, we first analyze the parsing and preprocessing phases of the approach. Then we describe the five components of our bug localization method. Finally, we show how these five components are combined to give us the final ranking of source files for each bug report.

3.1. Parsing and preprocessing phases

In the parsing phase, each source file is parsed to extract specific elements from them, such as comments and identifier names from classes, attributes, methods and variable declarations. Bug reports are also parsed to get their summary, description and submission date. We then extract stack traces from the description of the bug report if available.

After parsing the source files and bug reports, the preprocessing phase applies several steps to source files and bug reports to extract desired word tokens. First, a POS tagger is applied on the summary and description of the bug reports and comments of the source files to extract words with “NOUN” and “VERB” tags; we call these words the “POS tagged” tokens of each element. The complete form of these elements are also tokenized using white spaces and punctuation marks, and we keep them as a separate bag of words. These are referred to as the summary and description tokens.

In the next step we split the camel case and composite tokens of both source files and bug reports like “TreeView” into their individual tokens “Tree” and “Viewer”; we also keep the original compound form of these tokens. We then remove punctuation marks, numbers, a set of English stop words, and programming language keywords from tokens. Finally, the Porter stemming algorithm is used to reduce tokens to their stem; the unstemmed or full terms are also kept for the semantic similarity component.

3.2. Token matching component

Token matching component tries to boost the bug localization by finding exact matching tokens in specific parts of the bug report and source files. This component first checks for the exact appearance of a source file name in the summary of the bug report. If any summary tokens of the bug report are the exact source file name, then we assign a one-point score to that source file. If none of the summary tokens match any source file names, the component moves to the second stage that includes three steps as follows for each

source file with respect to each bug report:

1. Check if any of the bug report's POS tagged summary and description tokens match the tokens from the source file name, class names, and method names and assign the count of matched tokens as the score:

$$1st\ step\ score = count(matched\ tokens) \quad (1)$$

2. If zero tokens matched and no score was given in the previous step, check for the matches between the bug report's POS tagged tokens and the tokens in source file's comments. Then the count of common tokens minus the count of comments tokens is assigned as the score:

$$2nd\ step\ score = count(matched\ tokens) - count(comments\ tokens) \quad (2)$$

The subtraction part in the above equation adjusts the significance of the second step's score compared to the score of the first step. The matched tokens of the first step in most cases produce smaller positive integers compared to matched tokens from comments. However, matched tokens in the file name, class names and method names are more important than the matched tokens in comments, so we subtract the score of the second step by some count to make it less important.

3. If there was still no score and nothing matched, then check for the matches between the bug report's POS tagged tokens and the source file's attribute names. The score in this step is computed as:

$$3rd\ step\ score = count(matched\ tokens) - count(attribute\ tokens) \quad (3)$$

Here again like in step two, we adjust the importance of this score by subtracting the count of attribute tokens.

Finally, the scores given to source files regarding each bug report is normalized using a min-max normalizer to scale their range from 0 to 1. The normalization function of source file scores for each bug report defines as:

$$nor(x) = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (4)$$

where x is the set of original values, $nor(x)$ is the set of normalized values, x_{min} and x_{max} are the minimum and maximum value in our set of values.

We denote the score of this component as *TokenMatchingScore*.

3.3. VSM similarity component

We use a variation of the rVSM as proposed by Zhou et al. (2012) to find the similarity scores between bug reports and source files. We represent each bug report and source file as a vector of term weights. We use terms in the summary and description of bug reports to build their vectors. For source files, the combination of tokens from the file name, class names, method names, attribute names, and POS tagged tokens in comments are used. The weight $w_{i,d}$ of term i in document d is calculated by the term frequency-inverse document frequency (tf-idf) score:

$$w_{i,d} = (1 + \log(tf_{i,d})) \times \left(\log\left(\frac{\#src_files}{df_i}\right) + 1 \right) \quad (5)$$

In Eq. (5), $tf_{i,d}$ denotes the term frequency which is the number of occurrences of the term i in document d , $\#src_files$ is the total number of source files in the collection and df_i refers to the document frequency of the term i which is the number of documents that term appears in. the resulting tf-idf vectors are then normalized by the Euclidean norm:

$$V_{norm} = \frac{\vec{V}}{\sqrt{w_1^2 + w_2^2 + \dots + w_v^2}} \quad (6)$$

After computing the tf-idf values, we measure similarity scores between bug reports and source files using the cosine similarity of their vector representations as:

$$\cos(b, s) = \frac{\vec{V}_b \cdot \vec{V}_s}{|\vec{V}_b| \times |\vec{V}_s|} \quad (7)$$

where $\vec{V}_b$ and $\vec{V}_s$ are the vector of term weights for bug report b and source file s and the numerator is the dot product of these two vectors. $|\vec{V}_b|$ and $|\vec{V}_s|$ indicate the length of the vectors and is computed as follows for a vector of document d :

$$|\vec{V}_d| = \sqrt{w_{1,d}^2 + w_{2,d}^2 + \dots + w_{v,d}^2} \quad (8)$$

We also combine the source files' length scores with the result of the cosine similarity because larger source files tend to have a higher probability of having bugs (Zhang, 2009); Zhou et al. (2012) have experimented with different forms of length functions and have found out that the logistic form gives better overall MAP and MRR results.

$$LenScore_s(\#terms) = \frac{1}{1 + e^{-\lambda \times nor(\#terms)}} \quad (9)$$

Eq. (9) computes the length score of the source file s . The terms used in this function are the same ones used for building the source files' vectors (i.e., the tokens from file name, class names, method names, attribute names, and POS tagged tokens in comments), and $\#terms$ denotes the number of these terms in each source file that should be normalized using Eq. (4). The λ ($\lambda > 0$) parameter serves as a magnifier to adjust how much favor we give to larger files. By setting this parameter, we are able to get a better balance between favoring large files and reducing noise in large files (Wong et al., 2014). Throughout our experiments, we set λ to 12.

By multiplying Eqs. (7) and (9), the final formula for rVSM score will be as follows:

$$VSMSimilarityScore = \cos(b, s) \times LenScore_s(\#terms) \quad (10)$$

We compute this score between each bug report and all the source files to find the similarity relevance score between them. The ranking score of this component is denoted as *VSMSimilarityScore*.

3.4. Stack trace component

Some of the submitted bug reports may contain a stack trace that we can utilize to better locate the faulty source file. As shown in the introduction example, most of the time the faulty file is directly within the stack trace files. Each stack trace includes multiple stack frames, so we use a simple regular expression like `'at (.*)\((.*)\)'` to extract stack frames in the form of `'at package_name.class_name.method_name (file_name.java:line_number | Native Method | Unknown Source)'` from the description of bug reports.

The left side of each stack frame consists of the package name, class name, and the method name that were being executed when the reported bug has occurred. Inside parentheses of the right side can be in three different forms. The first form contains the name of the source file and the line number where the problem has occurred during the execution. We check these file names against the project's file names and remove stack frames that do not belong to the project's code base. This way we filter out wrong file names and external third-party libraries from the stack trace. The Second form is Native Method which shows that the stack frame is pointing to a low-level method that is implemented within the Java virtual machine (JVM). We remove these stack frames from the retrieved stack trace because they also don't belong to the source files of our target project. The last form is Unknown Source that normally relates to missing debug information (usually when users run the program using only JRE and not JDK or system is in release mode, and the debug mode is off). Since in Java, the name of a public class and its source file is the same, we check the class name from the left side of the stack frame against our project's file names. If a match is found, we use that class name as the file name for the Unknown Source stack frame.

After these processing steps and getting the appropriate stack frames from the bug report's stack trace, identical frames are merged into a single record considering their initial order. Schröter et al. (2010) showed that related source files for about 90% of bugs are located within the top 10 stack frames, and this probability decreases as we go through the stack trace from top to bottom. Therefore, we score source files based on the rank of each file in the stack trace as:

$$StackTraceScore = \begin{cases} \frac{1}{rank_s} & \text{if } s \text{ is in the stack trace} \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

where $rank_s$ is the rank of the source file s in the processed stack trace. This way higher files get better scores and the first file gets the complete score. In the process of scoring source files, we also take their package name into account to avoid false positives because of similar file names in different packages. The score of this component is indicated as *StackTraceScore*.

3.5. Semantic similarity component

While a bug report and its relevant source files usually share common terms, sometimes there are lexical mismatches between terms used in bug reports and their related source files (Ye et al., 2014). Lexical mismatch makes automatic bug localization harder since some domain knowledge about the project is then needed to locate relevant files.

To overcome this problem, we employ semantic similarity to help bridge the lexical gap between bug reports and their relevant source files. There are different approaches to achieve semantic similarity such as LSI (Manning, Raghavan, & Schütze, 2008), which uses the singular-value decomposition on a term-document matrix to reduce the co-occurrence matrix and transforms the original data into a much smaller dimensionality while preserving the semantic meanings of documents; or LDA, which is a generative statistical model and represents each document with a set of topics (Blei, Ng, & Jordan, 2003; Lukins et al., 2008). Semantic relations in lexical databases like WordNet (Miller, 1995) can also be used for computing semantic similarity (Liu, Zhou, & Zheng, 2007). However, in this component, word embedding and the dense vector representations of words is used. Word2Vec (Mikolov et al., 2013) and GloVe (Pennington et al., 2014) are two popular and widely used methods to construct vector representations for words. Word2Vec is a two-layer neural network and an unsupervised algorithm for learning the meaning behind words. It produces a dense vector space that gives each word in the corpus a vector such that similar words are located close to each other in that space. GloVe is also an unsupervised algorithm for constructing word vectors. It uses word occurrences in a corpus as its source of information and builds a word-word co-occurrence matrix and then employs matrix factorization to build word vectors.

We use 300-dimensional pre-trained GloVe word vectors that are trained on the Common Crawl corpus⁵ to build vectors for the bug reports and source files. Since pre-trained vectors are used, we use unstemmed or full term tokens of bug reports and source files in this component. For source files, tokens from file name, class names, attribute names, method names and comments are used. On the bug report's end, summary tokens and POS tagged tokens of the description is used. To build bug report and source file vectors we take the average of the vector of their tokens as follows:

$$\vec{V}_d = \frac{1}{N} \sum_{i=1}^N \vec{v}_i \quad (12)$$

where N is the total number of word vectors, $\vec{v}_i$ represents word vectors and $\vec{V}_d$ is the constructed document vector. Then, we apply cosine similarity between the bug report and source file vectors as shown in Eq. (7) to measure their semantic similarity:

$$\text{Semantic Similarity Score} = \cos(b, s) \quad (13)$$

The score of this component is denoted as *SemanticSimilarityScore*.

3.6. Fixed bug reports component

Previously fixed source files may be responsible for newer similar bugs because similar bugs tend to fix similar files (Murphy-Hill et al., 2013). Therefore, using similar previously fixed bug reports, we may find relevant buggy files associated with our new bug report. For each bug report, we use a multi-label classification algorithm only on its previously fixed bug reports to score relevant source files for that bug report.

We use terms in bug reports as the input and their fixed source files as the labels. Then, the multi-label learning algorithm outputs probability score of source files for a newly given bug report which represents the confidence of assigning these source files to the given bug report.

We extract features from terms of the bug reports to better utilize the multi-label learning algorithm. The summary of the bug reports is an important feature as users tend to say what the bug is about in a small number of concise words. We use the tf-idf weights of the preprocessed summary tokens of bug reports as a feature. We also add the tf-idf weights of the bug reports' POS tagged tokens as another feature. We then train an independent classifier for each label using a multinomial naive Bayes classifier as the base classifier for a one-vs.-the-rest (OvR) method. The different naive Bayes classifiers differ mainly by the assumptions they make regarding the distribution of $P(x_i | y)$ for a label variable y and a feature vector x_i . Multinomial naive Bayes implements the naive Bayes algorithm for multinomially distributed data and it has shown good performance for text classification (McCallum & Nigam, 1998). The distribution is parameterized by vectors $\theta_y = (\theta_{y1}, \dots, \theta_{yn})$ for each label y , where n is the number of features and θ_{yi} is the probability $P(x_i | y)$ of feature i appearing in a sample belonging to label y . The parameters θ_y are estimated by a smoothed version of maximum likelihood:

$$\theta_{yi} = \frac{N_{yi} + \alpha}{N_y + \alpha n} \quad (14)$$

where N_{yi} is the number of times feature i appears in a sample of label y and N_y is the total count of all features for label y . We use Laplace smoothing and set $\alpha = 1$ to prevent zero probabilities. The score of this component is denoted as *FixedBugReportsScore*:

$$\text{FixedBugReportsScore} = P(y) \times \prod_{i=1}^n P(x_i | y) \quad (15)$$

3.7. Combining scores

Now that we have the separate scores of our components, we combine them to get the final score and then rank source files based on that for each bug report. We use the linear combination of the scores of our components to compute the final score as follows:

$$\begin{aligned} \text{FinalScore} = & w_1 \times \text{TokenMatchingScore} \\ & + w_2 \times \text{VSMSimilarityScore} \\ & + w_3 \times \text{StackTraceScore} \\ & + w_4 \times \text{SemanticSimilarityScore} \\ & + w_5 \times \text{FixedBugReportsScore} \end{aligned} \quad (16)$$

where $w_1, w_2, w_3, w_4, w_5 \in [0, 1]$ indicate the different contribution weights of the token matching component, VSM similarity component, stack trace component, semantic similarity component, and fixed bug reports component to the final score. The value of these parameters can be determined empirically or in our dataset based on previously fixed bug reports for each bug report or by a grid search on a training data. This training data could be obtained from past manual bug localization efforts that are logged in bug repositories and version control systems (Wang et al., 2014). The goal here is to estimate these parameters the way they maximize an

⁵ <https://commoncrawl.org/>.

objective function. Our objective function uses two of the evaluation metrics and is defined as follows:

$$ObjFunc = MRR + MAP \quad (17)$$

Both MRR and MAP values (which will be further discussed in Section 4.2) are important since the former is used when a developer is interested to find at least one of the buggy files so that it can serve as an entry point in finding other buggy files, and the latter is used when a developer is interested in finding all the buggy files by examining the ranked list. We use the differential evolution (DE) algorithm (Storn & Price, 1997) to maximize the objective function and estimate the parameters. DE is a stochastic population-based method that is used for global optimization problems and can search large areas of candidate space. It iteratively tries to improve candidate solutions by mixing them together to create a trial candidate.

4. Experimental setup

In this section, first, we describe the dataset and evaluation metrics used to evaluate the performance of the proposed approach. Then, we discuss our research questions and present the implementation details.

4.1. Dataset

To evaluate the effectiveness of our proposed approach we use the benchmark dataset provided by Zhou et al. (2012), as various other studies also used the same dataset to evaluate their systems (Rahman et al., 2015; Wang & Lo, 2014; Saha et al., 2013; Wong et al., 2014; Zhou et al., 2017). The dataset contains bug reports of different open source projects and their associated source files. For each subject system, the already fixed bug reports are collected from its bug tracking system and then a heuristic is used to set the links between bug reports and source files. In our evaluation we use these three projects:

1. **AspectJ**:⁶ An aspect-oriented extension to the Java programming language. It uses Bugzilla as its bug tracking system.
2. **SWT**:⁷ A graphical widget toolkit for use with the Java platform. SWT also uses Bugzilla bug tracker.
3. **ZXing**:⁸ A barcode scanning library for Java and Android. ZXing previously used Google Code's issue tracking system, but it's now on GitHub and uses GitHub issue tracker.

Table 2 shows some statistics related to these projects such as the study period for the bug reports, the number of bug reports, the number of source files, and the number of reports containing stack traces.

4.2. Evaluation metrics

Our approach produces a ranked list of source files for each bug report, and we evaluate its performance using the following metrics:

- **Top@N**: This metric is the number of bug reports whose related fixed files are ranked in the top N ($N = 1, 5, 10$) of the returned results. For a bug report, if the top N results contain at least one file at which the bug should be fixed, we consider it a hit and the bug is located. The higher the value of this metric, the better the bug localization performance.
- **MRR**: Mean reciprocal rank is the average of the reciprocal ranks of a set of queries (Voorhees, 2001). The reciprocal rank (RR) of a query is the multiplicative inverse of the rank of the first relevant answer found. Here it measures the overall effectiveness of retrieval for a set of bug reports. Supposing we have a set of bug reports BR and the rank of the first relevant source file for bug report i is $first_i$, then MRR defines as:

$$MRR = \frac{1}{|BR|} \sum_{i=1}^{|BR|} \frac{1}{first_i} \quad (18)$$

The higher the MRR value, the better the bug localization performance.

- **MAP**: Mean average precision is a standard metric widely used in IR (Manning et al., 2008) that provides a metric to evaluate the average performance of locating all relevant files. In bug localization, a bug may be relevant to multiple files, and this metric considers all the relevant answers with their ranks for a single query. Supposing that m relevant source files are retrieved and ranked as $s_1, s_2, \dots, s_m$ for each bug report and having a set of bug reports BR , then the average precision (AP) and MAP define as:

$$MAP = \sum_{i=1}^{|BR|} \frac{AvgP(i)}{|BR|}, \quad AP = \frac{\sum_{k \in S} Prec@k}{m} \quad (19)$$

⁶ <https://eclipse.org/aspectj/>.

⁷ <https://www.eclipse.org/swt/>.

⁸ <https://github.com/zxing/zxing>.

Table 2

The statistics of the benchmark datasets.

Project	Study period	# Fixed bugs	# Source files	# Reports with stack trace
AspectJ	Jul 2002 – May 2010	286	6485	95 (33.2%)
SWT	Oct 2004 – Apr 2010	98	484	4 (4.1%)
ZXing	Mar 2010 – Sep 2010	20	391	1 (5%)

Here $Prec@k$ is the retrieval precision over the top k source files in the ranked list:

$$Prec@k = \frac{\# \text{ of relevant source files in top } k}{k} \quad (20)$$

The higher the MAP value, the better the bug localization performance.

4.3. Research questions

The main objective of our experiments is to evaluate the performance of the proposed approach and its components. We aim to answer the following research questions:

RQ1: How is the overall performance of the proposed approach and does it outperform other bug localization approaches?

RQ2: How effective is the approach without the previously fixed bug reports component and how does it compare to other approaches in the same situation?

RQ3: How accurate are the individual components of the proposed approach?

4.4. Analysis procedure

To answer the defined RQs, we apply our approach to the three sets of bug reports for the projects in our dataset. The returned ranked list of source files is then evaluated, and the performance of the approach is measured by means of Top@N, MRR, and MAP metrics. For RQ1, our approach is compared to BugLocator (Zhou et al., 2012), BLUIR (Saha et al., 2013), BRTracer (Rahman et al., 2015; Wong et al., 2014), Rahman et al. (2015) approach, and AmaLgam (Wang & Lo, 2014). We also perform a Wilcoxon signed-rank test for the statistical comparison of the approaches. For RQ2, we apply our approach without the fixed bug reports component and compare its results to BugLocator, BLUIR, BRTracer, and Zhou et al. (2017) in the same situation. The fixed bug reports component is compared individually to the Usual Suspects baseline algorithm proposed by Kim, Tao, Kim, and Zeller (2013). For RQ3, the different components of the proposed approach are separately applied to the bug reports to find their accuracy.

4.5. Implementation details

Our approach is implemented in the Python programming language. The first step in our approach is parsing the source files and bug reports. We use the combination of javalang parser⁹ and the lexers available in Pygments library¹⁰ to extract identifiers and comments from source files. Pygments is actually a syntax highlighting package that has a wide range of lexers for different languages and text formats to do its job, and they can be used independently to lexically analyze a source file. In the preprocessing phase, NLTK¹¹ is used for POS tagging, stop word removal and stemming. NLTK is a powerful text processing library for classification, tokenization, stemming, tagging, parsing, semantic reasoning, and has interfaces for different corpus and lexical resources.

To build tf-idf vectors for bug reports and source files, we employ the vectorizer available in scikit-learn (Pedregosa et al., 2011). Scikit-learn is a machine learning library that features various algorithms and tools for different data mining and data analysis tasks. We also utilize the multinomial naive Bayes and OvR implementation of scikit-learn to construct the fixed bug reports component.

Finally, the word vectors of GloVe Common Crawl model in Spacy¹² is used to compute the semantic similarity, and the parameter estimation is done via the DE implementation in SciPy.¹³ The complete data and source code of our approach is available online.¹⁴

⁹ <https://github.com/c2nes/javalang>.

¹⁰ <http://pygments.org/>.

¹¹ <http://www.nltk.org/>.

¹² <https://spacy.io/>.

¹³ <https://scipy.org/>.

¹⁴ <https://github.com/h4iku/bug-localization>.

Table 3

The overall performance of our approach.

Project	Approach	Top@1	Top@5	Top@10	MRR	MAP
AspectJ	Our approach	133 (46.5%)	192 (67.1%)	212 (74.1%)	0.56	0.32
	Amalgam	127 (44.4%)	187 (65.4%)	209 (73.1%)	0.54	0.33
	Rahman et al.	N/A	N/A	N/A	N/A	N/A
	BRTracer	113 (39.5%)	173 (60.5%)	197 (68.9%)	0.49	0.29
	BLUIR	97 (33.9%)	150 (52.4%)	176 (61.5%)	0.43	0.25
	BugLocator	88 (30.8%)	146 (51.0%)	170 (59.4%)	0.41	0.22
SWT	Our approach	67 (68.4%)	84 (85.7%)	88 (89.8%)	0.76	0.65
	Amalgam	61 (62.2%)	80 (81.6%)	88 (89.8%)	0.71	0.62
	Rahman et al.	47 (48.0%)	70 (71.4%)	81 (82.7%)	0.60	0.54
	BRTracer	46 (46.9%)	78 (79.6%)	87 (88.8%)	0.59	0.53
	BLUIR	55 (56.1%)	75 (76.5%)	86 (87.8%)	0.66	0.58
	BugLocator	39 (39.8%)	66 (67.3%)	80 (81.6%)	0.53	0.45
ZXing	Our approach	12 (60.0%)	15 (75.0%)	16 (80.0%)	0.67	0.56
	Amalgam	8 (40.0%)	13 (65.0%)	14 (70.0%)	0.51	0.41
	Rahman et al.	9 (45.0%)	14 (70.0%)	15 (75.0%)	0.55	0.50
	BRTracer	10 (50.0%)	13 (65.0%)	15 (75.0%)	0.61	0.49
	BLUIR	8 (40.0%)	13 (65.0%)	14 (70.0%)	0.49	0.39
	BugLocator	8 (40.0%)	12 (60.0%)	14 (70.0%)	0.50	0.44

5. Results and discussion

5.1. Overall results

Table 3 shows the experimental results on AspectJ, SWT and ZXing projects and also the comparison with the reported results of the five state-of-the-art bug localization approaches including BugLocator (Zhou et al., 2012), BLUIR (Saha et al., 2013), BRTracer (Rahman et al., 2015; Wong et al., 2014), Rahman et al. (2015), and Amalgam (Wang & Lo, 2014). Since these approaches all have used the same dataset as us, we compare our results with their performance as it's reported in their papers. Also, Rahman et al. (2015) haven't reported its results for AspectJ; therefore we can't compare our results against their approach for this project.

Results show that except for Amalgam's MAP value on AspectJ, our approach outperforms or equals all other approaches on all metrics for all the evaluated projects. We should also consider that Amalgam employs version histories of source files as an additional information source, which we don't use. Furthermore, all the approaches use previously fixed bug reports to improve their performance, but they use a simple VSM similarity for this matter while we employ a different method to deal with similar bug reports and use a multi-label classification technique to get probability scores for relevant source files. We should note that in all cases, the proposed approach improves MRR value which shows that a developer needs to go through fewer files in a ranked list to find the one that needs to be fixed.

Considering AspectJ, our approach significantly improves all metrics compared to BugLocator, BLUIR, and BRTracer. Regarding MRR and MAP values, it achieves 0.56 and 0.32, respectively. There are 133 (46.5%), 192 (67.1%), and 212 (74.1%) bugs located in top 1, top 5, and top 10 respectively using the proposed approach. Compared to Amalgam, our approach achieves about 1% less MAP score, but it has an improvement of 3.7% for MRR value.

For SWT, our approach achieves 67 (68.4%), 84 (85.7%) and 88 (89.8%) successfully located bugs in top 1, top 5, and top 10, respectively. It improves all metrics compared to other approaches except for Top@10 metric compared to Amalgam where they equally locate source files for 88 (89.8%) bug reports.

For the ZXing project, our approach significantly performs better than other approaches in all evaluation metrics. We have achieved 80% accuracy in top 10 for ZXing and could locate relevant source files of 16 bug reports among its 20 bug reports where no other approaches have achieved this score. The proposed approach improves MAP value by 27.3%, 43.6%, 14.3%, 12.0%, and 36.6% over BugLocator, BLUIR, BRTracer, Rahman et al. (2015) and Amalgam, respectively. These results show that our approach works well even on smaller projects with not much historical data.

5.1.1. Statistical testing

We perform statistical testing to check if the improvement we get over other approaches is significant. Since only tools for BugLocator and BRTracer were available, we present results for these two approaches. We use the Wilcoxon signed-rank test (Wilcoxon, 1945) which is a non-parametric test that makes no assumptions about the distribution of the data. Wilcoxon signed-rank test is robust, widely used in literature and is preferred when the assumption of data normality may not be necessarily true. We use this test to find the significance of improvement regarding MRR and MAP. In fact, the RR and AP values for each bug report achieved by our approach in each project is compared to values achieved by BugLocator and BRTracer. The two-tailed statistical significance is measured at $p \leq 0.05$ and based on the values shown in Table 4, in terms of both MRR and MAP, the statistical improvement of our approach over both BugLocator and BRTracer is significant for all projects.

Table 4

Wilcoxon signed-rank test comparison in terms of MRR and MAP.

Project	BugLocator MRR	MAP	BRTracer MRR	MAP
AspectJ	$p < 0.001$, $Z = -6.883$	$p < 0.001$, $Z = -4.248$	$p < 0.001$, $Z = -4.933$	$p < 0.001$, $Z = -5.660$
SWT	$p < 0.001$, $Z = -5.576$	$p < 0.001$, $Z = -4.970$	$p < 0.001$, $Z = -4.062$	$p < 0.001$, $Z = -3.585$
ZXing	$p = 0.003$, $Z = -3.015$	$p = 0.002$, $Z = -3.055$	$p = 0.007$, $Z = -2.695$	$p = 0.009$, $Z = -2.606$

5.2. Results without previously fixed bug reports and fixed bug reports component comparison

All the other approaches examine bugs that have been fixed before to adjust the rankings of the relevant files. Our approach also uses these previously fixed bug reports to improve its effectiveness but the historical data and already resolved bug reports may not always be available for all software projects; therefore we also evaluate the performance of the proposed approach without the fixed bug reports component. Since AmaLgam and Rahman et al. (2015) haven't reported their performance without using similar reports, we can't compare our results with them, but instead, we add a new approach by Zhou et al. (2017) to the compared approaches. They present an approach that only uses bug reports and source files of a software project, and it doesn't rely on previous bug reports. They propose an adaptive method that uses different ranking strategies for top 1 and top N recommendations. They use a parameter called “opt” to alternate between these two strategies. Because we are comparing the complete ranking of source files against other approaches, we use the results of “opt = false” strategy of their method that provides a list of faulty files instead of only one faulty file. Table 5 shows the results of our approach without the fixed bug reports component compared to other systems in the same situation.

These results show that our approach outperforms all compared approaches without using previous bug reports. It has significant improvement in all evaluation metrics and locates 200 (69.9%), 88 (89.8%), and 15 (75.0%) bugs for AspectJ, SWT, and ZXing projects in top 10 recommended source files, respectively. It only locates about 3.2% fewer bugs compared to when using fixed bug reports, but fixed bug reports component has more impact on better overall ranking of the source files. In this situation, the proposed approach, on average achieves improvement of 49.0%, 23.7%, 21.7%, and 37.7% over BugLocator, BLUiR, BRTrace, and Zhou et al. (2017) in terms of MAP, respectively. It also improves the MRR in the same order by 50.0%, 23.9%, 22.3%, and 43.3%.

We also compare the performance of the fixed bug reports component against a baseline proposed by Kim et al. (2013) known as the Usual Suspects that ranks source files based on how many times they are fixed. We reimplemented the baseline, and it ranks source files for each bug report based on its previously fixed bug reports. This is the same way the fixed bug reports component uses the bug reports dataset to compute probabilities for each bug report. Table 6 shows the results of this comparison; the “Frequently fixed” rows are the performance of the baseline. We can see that our fixed bug reports component, which uses a multi-label classification algorithm, performs much better than the baseline algorithm. Moreover, the improvement for the ZXing project is not too much, and the results are close because its dataset is small and it only contains 20 bug reports. The fair results of the baseline algorithm show that it is often the case that relevant files are the files that have frequently been fixed.

5.3. Separate component results

In this subsection, we present the results for each component of the proposed approach. Table 7 shows these results. The VSM similarity component, which uses rVSM, has the best performance across all projects among all components. The token matching component gives us a great boost toward better performance. The token matching component works better in SWT and ZXing projects than in AspectJ. This happens due to the difference in the nature of projects. In AspectJ, the matching between bug report terms and

Table 5

The performance of our approach without using previously fixed bug reports.

Project	Approach	Top@1	Top@5	Top@10	MRR	MAP
AspectJ	Our approach	130 (45.5%)	182 (63.6%)	200 (69.9%)	0.54	0.31
	Y. Zhou et al. (opt = false)	76 (26.6%)	135 (47.2%)	168 (58.7%)	0.37	0.21
	BRTracer	111 (38.8%)	168 (58.7%)	191 (66.8%)	0.48	0.27
	BLUiR	92 (32.2%)	146 (51.0%)	173 (60.5%)	0.41	0.24
	BugLocator	65 (22.7%)	117 (40.9%)	159 (55.6%)	0.33	0.17
SWT	Our approach	63 (64.3%)	85 (86.7%)	88 (89.8%)	0.75	0.63
	Y. Zhou et al. (opt = false)	39 (39.8%)	72 (73.5%)	81 (82.6%)	0.55	0.49
	BRTracer	37 (37.8%)	73 (74.5%)	80 (81.6%)	0.54	0.47
	BLUiR	54 (55.1%)	75 (76.5%)	85 (86.7%)	0.65	0.56
	BugLocator	31 (31.6%)	64 (65.3%)	76 (77.6%)	0.47	0.40
ZXing	Our approach	11 (55.0%)	15 (75.0%)	15 (75.0%)	0.63	0.52
	Y. Zhou et al. (opt = false)	6 (30.0%)	13 (65.0%)	13 (65.0%)	0.42	0.36
	BRTracer	9 (45.0%)	14 (70.0%)	15 (75.0%)	0.55	0.46
	BLUiR	8 (40.0%)	13 (65.0%)	14 (70.0%)	0.49	0.38
	BugLocator	8 (40.0%)	11 (55.0%)	14 (70.0%)	0.48	0.41

Table 6

Comparison of the fixed bug reports component and the frequently fixed approach.

Project	Approach	Top@1	Top@5	Top@10	MRR	MAP
AspectJ	Fixed bug reports component	40 (14.0%)	94 (32.9%)	122 (42.7%)	0.24	0.11
	Frequently fixed	14 (4.9%)	57 (19.9%)	96 (33.6%)	0.13	0.06
SWT	Fixed bug reports component	26 (26.5%)	42 (42.9%)	48 (49.0%)	0.35	0.28
	Frequently fixed	9 (9.2%)	31 (31.6%)	40 (40.8%)	0.20	0.14
ZXing	Fixed bug reports component	2 (10.0%)	4 (20.0%)	7 (35.0%)	0.15	0.13
	Frequently fixed	1 (5.0%)	3 (15.0%)	6 (30.0%)	0.12	0.10

Table 7

The performance of different components of our approach.

Project	Component	Top@1	Top@5	Top@10	MRR	MAP
AspectJ	Token matching	8 (2.8%)	24 (8.4%)	33 (11.5%)	0.05	0.03
	VSM similarity	90 (31.5%)	150 (52.4%)	179 (62.6%)	0.42	0.23
	Stack trace	56 (19.6%)	66 (23.1%)	66 (23.1%)	0.21	0.11
	Semantic similarity	25 (8.7%)	55 (19.2%)	81 (28.3%)	0.15	0.09
	Fixed bug reports	40 (14.0%)	94 (32.9%)	122 (42.7%)	0.24	0.11
	Combined	133 (46.5%)	192 (67.1%)	212 (74.1%)	0.56	0.32
SWT	Token matching	39 (39.8%)	70 (71.4%)	71 (72.4%)	0.54	0.43
	VSM similarity	42 (42.9%)	79 (80.6%)	85 (86.7%)	0.59	0.51
	Stack trace	8 (8.2%)	8 (8.2%)	8 (8.2%)	0.09	0.04
	Semantic similarity	15 (15.3%)	35 (35.7%)	44 (44.9%)	0.25	0.22
	Fixed bug reports	26 (26.5%)	42 (42.9%)	48 (49.0%)	0.35	0.28
	Combined	67 (68.4%)	84 (85.7%)	88 (89.8%)	0.76	0.65
ZXing	Token matching	5 (25.0%)	11 (55.0%)	12 (60.0%)	0.38	0.30
	VSM similarity	7 (35.0%)	14 (70.0%)	15 (75.0%)	0.49	0.42
	Stack trace	1 (5.0%)	3 (15.0%)	5 (25.0%)	0.12	0.11
	Semantic similarity	3 (15.0%)	9 (45.0%)	13 (65.0%)	0.32	0.24
	Fixed bug reports	2 (10.0%)	4 (20.0%)	7 (35.0%)	0.15	0.13
	Combined	12 (60.0%)	15 (75.0%)	16 (80.0%)	0.67	0.56

identifiers in certain places of source files has not occurred much. The stack trace component localizes relevant source files for 56 (19.6%) bug reports in top 1 for AspectJ which shows that usually, the buggy file is directly within the stack trace. However, it only localizes eight and one bugs for SWT and ZXing in top 1, respectively. That's because only four bug reports in SWT and one bug report in the ZXing dataset contains a stack trace. This also shows that some of the relevant source files for bug reports of these projects are actually located based on the natural order of source files in the dataset since source files for bug reports without a stack trace and those that do not appear in the stack trace, get no scores from the stack trace component. The semantic similarity component has fair results for all projects, but it's not high compared to VSM similarity component. This is because, in addition to correct target source files, it also retrieves many source files that are not relevant to a given bug report. However, regarding covering lexical mismatches, it does its job well and achieving high accuracy is not the case for this component. The fixed bug reports component, which uses the previously fixed bugs, achieves better accuracy for AspectJ and SWT projects than for ZXing, and it's merely because of the small number of bug reports available for this project. Finally, it can be seen that the combined component always has the best results for each project.

We also want to see how these components combine and contribute to the final score. Table 8 shows the average contribution weights of each component estimated by the DE algorithm for each software project. The stack trace component has the most contribution weight for AspectJ and ZXing projects and also a high weight for SWT, albeit, this doesn't mean that this component has best results. This is because located source files are usually correct in this component and other source files have no score, so adding those to the scores of other components doesn't affect anything. VSM similarity component is the next best contributor for AspectJ and SWT projects, and the semantic similarity component plays the same role for the ZXing project since these components supposed to act complementary to each other. Lastly, there are fixed bug reports and token matching components that boost the system toward more accurate source files. Based on non-zero estimated contribution weights of the components, we can see that all the components have a positive impact on the final score and none of them are redundant for this matter.

Table 8

The contribution weights of each component.

Project	w_1	w_2	w_3	w_4	w_5
AspectJ	0.038	0.616	0.983	0.186	0.344
SWT	0.340	0.783	0.612	0.664	0.159
ZXing	0.045	0.090	0.921	0.647	0.047

5.4. Applicability of the approach

We evaluated our approach using different evaluation metrics in previous subsections, but various other evaluating dimensions can be considered when dealing with ranked list results of a bug localization system (Avazpour, Pitakrat, Grunske, & Grundy, 2014). In this subsection, we look from the utility point of view and discuss the applicability of our bug localization approach.

Textual bug localization approaches usually integrate into issue trackers to be next to their primary source of debugging data. Given a new bug report, our system can be used to recommend a ranked list of relevant source files for that bug. The success of a bug localization approach depends much on the quality of bug reports and the queries it forms from them. Formulating a query that can accurately retrieve the relevant source files is a challenging task for developers as it requires a certain level of knowledge and experience regarding the project's code base (Sisman & Kak, 2013). Our approach helps in designing useful queries to find the faulty file quicker. However, it doesn't compromise the flexibility in dealing with the bug since it leaves freedom to choose how to do it, and developers can still use whatever tool their development environment provides to find the faulty file (Borg, Alégroth, & Runeson, 2017).

It has been shown that using a bug localization tool helps save debugging time, although the amount of time it saves, depends on the quality of the bug report, the complexity of the bug itself, and the developers' experience (Wang et al., 2015). Developers usually do not find a debugging tool useful if they cannot find the appropriate source files early in the resulting ranked list. Experts are faster in locating the buggy file when using a bug localization tool (Parnin & Orso, 2011). Moreover, using better bug localization approaches can shorten the amount of time needed to find the faulty source file (Xia, Bao, Lo, & Li, 2016).

We also conducted a small experiment using six developers to test the time-saving applicability of our approach. The developers participated in the experiment were unfamiliar with the given project. We divided them into two groups and gave each group a bug report and source files of the project; the second group was given a ranked list of source files suggested by our approach as well. We generated the ranked list of source files using our tool for the given bug report and then gave the results to them in a text file. The chosen bug report was neither too easy nor too difficult to fix. We then asked both groups to find the fault in the source files based on the given bug report. We calculated the time elapsed from the moment a developer started to the moment he or she located the bug in the correct faulty source file. On average it took the first group 12 min to find where the problem was and the other group found it in 5 min. This good time boost is because the faulty source file was ranked as the second item in the resulting ranked list. Developers usually accept inspecting more or less five items and a lot of them agree that inspecting more than ten items is beyond their acceptability level (Kochhar, Xia, Lo, & Li, 2016).

5.5. Threats to validity

There are several threats that might affect the validity of our findings. The internal validity relates to experimental errors and the implementation of our approach. To reduce this threat, we compared our results with several existing bug localization approaches (Rahman et al., 2015; Saha et al., 2013; Wang & Lo, 2014; Wong et al., 2014; Zhou et al., 2012, 2017) using the same dataset and evaluation metrics as theirs to be sure that the improvement is only because of our approach. We have also double checked the dataset and our implementation, but there may still be errors that we have not noticed.

The external validity concerns the generalizability of our results. Like almost all textual bug localization approaches, our approach also relies on the quality of the bug reports and good programming practices in naming classes, methods, and variables. Therefore, the quality of bug reports, the identifier naming conventions and the way comments are written in source files are a threat to external validity. Poorly written bug reports and non-meaningful identifier names can affect the performance of our approach. To mitigate this threat, we experimented with bug reports of three open source projects with different sizes that are developed independently. The projects we analyzed were all open source and written in Java. Proprietary applications and their bug reports can be different from the open source ones, and this may impact the performance of our approach. From the language point of view, although the projects were all written in Java, the principles of our approach can be applied to other object-oriented languages as well. Still, in future work, we plan to further reduce these threats by analyzing more open source and proprietary projects written in different programming languages.

The construct validity refers to the suitability of the metrics used in our experimental evaluation. We used three different metrics including Top@N, MRR, and MAP. These metrics are standard performance measures that are extensively used in related and previous research studies to assess the performance of their methods. These metrics sufficiently captured the different aspects of performance in our approach and there is little threat to construct validity.

6. Conclusion and future work

The developer teams of a software project need to locate appropriate files for each received bug report to solve the system's problem. For this matter, searching through a considerable amount of source files is inefficient and assigning bug reports to the wrong developers wastes their time. To overcome this manual effort, bug localization techniques try to automate the process and rank relevant source files for each bug report. This way buggy files will rank higher, and the developer can find them faster.

In this paper, we have proposed a five-component bug localization approach. We have extracted specific word tokens from bug reports and source files and used similarity approaches to better locate faulty source files. We have also used a multi-label classification algorithm on previously fixed bug reports and the information from stack traces to improve the results. Our approach has evaluated on the data of three open source software projects, and the results showed improvements in ranking scores and also MRR

and MAP values. It can rank related source files for more than 52% of bugs by recommending only one source file and 78% by recommending ten files.

In future work, we will try to improve the performance of our approach's components. We can add new features in the system and also use different properties from the domain knowledge of software projects like building a graph representation of different associations in software projects such as the association between developers and their source files. The relation between different source files, the impact analysis of software changes, and utilization of the historical data available in software repositories can be used to further enhance the performance.

References

- Abreu, R., Zoetewij, P., Golsteijn, R., & van Gemund, A. J. C. (2009). A practical evaluation of spectrum-based fault localization. *Journal of Systems and Software*, 82(11), 1780–1792. <https://doi.org/10.1016/j.jss.2009.06.035>.
- Anvik, J., Hiew, L., & Murphy, G. C. (2005). Coping with an open bug repository. *Proceedings of the 2005 OOPSLA workshop on eclipse technology eXchange* (pp. 35–39). New York, NY: ACM. <https://doi.org/10.1145/1117696.1117704>.
- Avazpour, I., Pitakrat, T., Grunske, L., & Grundy, J. (2014). *Dimensions and metrics for evaluating recommendation systems*. *Recommendation systems in software engineering*. Heidelberg, Berlin: Springer 245–273. https://doi.org/10.1007/978-3-642-45135-5_10.
- Bavota, G. (2016). Mining unstructured data in software repositories: Current and future trends. *23rd international conference on software analysis, evolution, and reengineering*. 5. *23rd international conference on software analysis, evolution, and reengineering* (pp. 1–12). Piscataway, NJ: IEEE. <https://doi.org/10.1109/SANER.2016.47> (SANER).
- Bettenburg, N., Just, S., Schröter, A., Weiss, C., Premraj, R., & Zimmermann, T. (2008). What makes a good bug report? *Proceedings of the 16th ACM SIGSOFT international symposium on foundations of software engineering* (pp. 308–318). New York, NY: ACM. <https://doi.org/10.1145/1453101.1453146>.
- Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet allocation. *Journal of Machine Learning Research*, 3(Jan), 993–1022.
- Borg, M., Alégroth, E., & Runeson, P. (2017). Software engineers' information seeking behavior in change impact analysis - An interview study. *25th IEEE/ACM international conference on program comprehension (ICPC)* (pp. 12–22). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ICPC.2017.20>.
- Borg, M., Runeson, P., & Ardo, A. (2014). Recovering from a decade: A systematic mapping of information retrieval approaches to software traceability. *Empirical Software Engineering*, 19(6), 1565–1616. <https://doi.org/10.1007/s10664-013-9255-y>.
- Borg, M., Wnuk, K., Regnell, B., & Runeson, P. (2017). Supporting change impact analysis using a recommendation system: An industrial case study in a safety-critical context. *IEEE Transactions on Software Engineering*, 43(7), 675–700. <https://doi.org/10.1109/TSE.2016.2620458>.
- Cleland-Huang, J., Gotel, O. C. Z., Huffman Hayes, J., Mäder, P., & Zisman, A. (2014). *Software traceability: Trends and future directions*. *Future of software engineering*. New York, NY: ACM 55–69. <https://doi.org/10.1145/2593882.2593891>.
- Dit, B., Revelle, M., Gethers, M., & Poshvanyk, D. (2013). Feature location in source code: A taxonomy and survey. *Journal of Software: Evolution and Process*, 25(1), 53–95. <https://doi.org/10.1002/smr.567>.
- Eddy, B. P., Kraft, N. A., & Gray, J. (2018). Impact of structural weighting on a latent Dirichlet allocation-based feature location technique. *Journal of Software: Evolution and Process*, 30(1), <https://doi.org/10.1002/smr.1892>.
- Gay, G., Haiduc, S., Marcus, A., & Menzies, T. (2009). On the use of relevance feedback in IR-based concept location. *2009 IEEE international conference on software maintenance* (pp. 351–360). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ICSM.2009.5306315>.
- Gharibi, R., Rasekh, A. H., & Sadreddini, M. H. (2017). Locating relevant source files for bug reports using textual analysis. *2017 international symposium on computer science and software engineering conference (CSSE)* (pp. 67–72). Piscataway, NJ: IEEE. <https://doi.org/10.1109/CSSE.2017.8320119>.
- Jeong, G., Kim, S., & Zimmermann, T. (2009). Improving bug triage with bug tossing graphs. *Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on the foundations of software engineering* (pp. 111–120). New York, NY: ACM. <https://doi.org/10.1145/1595696.1595715>.
- Jiang, H., Nazar, N., Zhang, J., Zhang, T., & Ren, Z. (2017). PRST: A pagerank-based summarization technique for summarizing bug reports with duplicates. *International Journal of Software Engineering and Knowledge Engineering*, 27(06), 869–896. <https://doi.org/10.1142/S0218194017500322>.
- Jones, J. A., & Harrold, M. J. (2005). Empirical evaluation of the tarantula automatic fault-localization technique. *Proceedings of the 20th IEEE/ACM international conference on automated software engineering* (pp. 273–282). New York, NY: ACM. <https://doi.org/10.1145/1101908.1101949>.
- Jonsson, L., Borg, M., Broman, D., Sandahl, K., Eldh, S., & Runeson, P. (2016). Automated bug assignment: Ensemble-based machine learning in large scale industrial contexts. *Empirical Software Engineering*, 21(4), 1533–1578. <https://doi.org/10.1007/s10664-015-9401-9>.
- Khatiwada, S., Tushev, M., & Mahmoud, A. (2018). Just enough semantics: An information theoretic approach for IR-based software bug localization. *Information and Software Technology*, 93, 45–57. <https://doi.org/10.1016/j.infsof.2017.08.012>.
- Kim, D., Tao, Y., Kim, S., & Zeller, A. (2013). Where should we fix this bug? A two-phase recommendation model. *IEEE Transactions on Software Engineering*, 39(11), 1597–1610. <https://doi.org/10.1109/TSE.2013.24>.
- Kim, S., Zimmermann, T., Whitehead Jr., E. J., & Zeller, A. (2007). Predicting faults from cached history. *Proceedings of the 29th international conference on software engineering* (pp. 489–498). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ICSE.2007.66>.
- Kochhar, P. S., Xia, X., Lo, D., & Li, S. (2016). Practitioners' expectations on automated fault localization. *Proceedings of the 25th international symposium on software testing and analysis* (pp. 165–176). New York, NY: ACM. <https://doi.org/10.1145/2931037.2931051>.
- Lam, A. N., Nguyen, A. T., Nguyen, H. A., & Nguyen, T. N. (2015). Combining deep learning with information retrieval to localize buggy files for bug reports. *30th IEEE/ACM international conference on automated software engineering (ASE)* (pp. 476–481). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ASE.2015.73>.
- Le, T.-D. B., Lo, D., Le Goues, C., & Grunske, L. (2016). A learning-to-rank based fault localization approach using likely invariants. *Proceedings of the 25th international symposium on software testing and analysis* (pp. 177–188). New York, NY: ACM. <https://doi.org/10.1145/2931037.2931049>.
- Le, T.-D. B., Oentaryo, R. J., & Lo, D. (2015). Information retrieval and spectrum based bug localization: Better together. *Proceedings of the 10th joint meeting on foundations of software engineering* (pp. 579–590). New York, NY: ACM. <https://doi.org/10.1145/2786805.2786880>.
- Le, T.-D. B., Thung, F., & Lo, D. (2014). Predicting effectiveness of IR-based bug localization techniques. *25th IEEE international symposium on software reliability engineering* (pp. 335–345). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ISSRE.2014.39>.
- Liu, C., Fei, L., Yan, X., Han, J., & Midkiff, S. P. (2006). Statistical debugging: A hypothesis testing-based approach. *IEEE Transactions on Software Engineering*, 32(10), 831–848. <https://doi.org/10.1109/TSE.2006.105>.
- Liu, X.-Y., Zhou, Y.-M., & Zheng, R.-S. (2007). Measuring semantic similarity in WordNet. *2007 international conference on machine learning and cybernetics*. 6. *2007 international conference on machine learning and cybernetics* (pp. 3431–3435). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ICMLC.2007.4370741>.
- Lukins, S. K., Kraft, N. A., & Etzkorn, L. H. (2008). Source code retrieval for bug localization using latent Dirichlet allocation. *Proceedings of the 15th working conference on reverse engineering* (pp. 155–164). Piscataway, NJ: IEEE. <https://doi.org/10.1109/WCRE.2008.33>.
- Mani, S., Catherine, R., Sinha, V. S., & Dubey, A. (2012). Ausum: Approach for unsupervised bug report summarization. *Proceedings of the ACM SIGSOFT 20th international symposium on the foundations of software engineering* (pp. 1–11). New York, NY: ACM. <https://doi.org/10.1145/2393596.2393607>.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.
- Marcus, A., Sergeyev, A., Rajlich, V., & Maletic, J. I. (2004). An information retrieval approach to concept location in source code. *11th working conference on reverse engineering* (pp. 214–223). Piscataway, NJ: IEEE. <https://doi.org/10.1109/WCRE.2004.10>.
- McCallum, A., & Nigam, K. (1998). A comparison of event models for naive bayes text classification. *AAAI-98 workshop on learning for text categorization*. 752. *AAAI-98 workshop on learning for text categorization* (pp. 41–48). Madison: WI.

- Mehta, P., & Majumder, P. (2018). Effective aggregation of various summarization techniques. *Information Processing & Management*, 54(2), 145–158. <https://doi.org/10.1016/j.ipm.2017.11.002>.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. In C. J. C. Burges, L. Bottou, M. Welling, Z. Ghahramani, & K. Q. Weinberger (Eds.), *Advances in neural information processing systems* (pp. 3111–3119). New York City, NY: Curran Associates, Inc. Retrieved from <http://papers.nips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality.pdf>.
- Miller, G. A. (1995). Wordnet: A lexical database for english. *Communications of the ACM*, 38(11), 39–41. <https://doi.org/10.1145/219717.219748>.
- Moreno, L., Treadway, J. J., Marcus, A., & Shen, W. (2014). On the use of stack traces to improve text retrieval-based bug localization. *2014 IEEE international conference on software maintenance and evolution* (pp. 151–160). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ICSME.2014.37>.
- Murphy-Hill, E., Zimmermann, T., Bird, C., & Nagappan, N. (2013). The design of bug fixes. *Proceedings of the 2013 international conference on software engineering* (pp. 332–341). Piscataway, NJ: IEEE. Retrieved from <http://dl.acm.org/citation.cfm?id=2486788.2486833>.
- Nguyen, A. T., Nguyen, T. T., Al-Kofahi, J., Nguyen, H. V., & Nguyen, T. N. (2011). A topic-based approach for narrowing the search space of buggy files from a bug report. *Proceedings of the 26th IEEE/ACM international conference on automated software engineering* (pp. 263–272). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ASE.2011.6100062>.
- Parnin, C., & Orso, A. (2011). Are automated debugging techniques actually helping programmers? *Proceedings of the 2011 international symposium on software testing and analysis* (pp. 199–209). New York, NY: ACM. <https://doi.org/10.1145/2001420.2001445>.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(Oct), 2825–2830.
- Pennington, J., Socher, R., & Manning, C. (2014). Glove: Global vectors for word representation. *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)* (pp. 1532–1543).
- Poshyvanyk, D., Gueheneuc, Y. G., Marcus, A., Antoniol, G., & Rajlich, V. (2007). Feature location using probabilistic ranking of methods based on execution scenarios and information retrieval. *IEEE Transactions on Software Engineering*, 33(6), 420–432. <https://doi.org/10.1109/TSE.2007.1016>.
- Rahman, S., Ganguly, K. K., & Sakib, K. (2015). An improved bug localization using structured information retrieval and version history. *18th international conference on computer and information technology (ICCIT)* (pp. 190–195). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ICCITech.2015.7488066>.
- Rajlich, V., & Wilde, N. (2002). The role of concepts in program comprehension. *Proceedings 10th international workshop on program comprehension* (pp. 271–278). Piscataway, NJ: IEEE. <https://doi.org/10.1109/WPC.2002.1021348>.
- Rao, S., & Kak, A. (2011). Retrieval from software libraries for bug localization: A comparative study of generic and composite text models. *Proceedings of the 8th working conference on mining software repositories* (pp. 43–52). New York, NY: ACM. <https://doi.org/10.1145/1985441.1985451>.
- Rastkar, S., Murphy, G. C., & Murray, G. (2014). Automatic summarization of bug reports. *IEEE Transactions on Software Engineering*, 40(4), 366–380. <https://doi.org/10.1109/TSE.2013.2297712>.
- Runeson, P., Alexandersson, M., & Nyholm, O. (2007). Detection of duplicate defect reports using natural language processing. *Proceedings of the 29th international conference on software engineering* (pp. 499–510). Washington, DC: IEEE Computer Society. <https://doi.org/10.1109/ICSE.2007.32>.
- Saha, R. K., Lease, M., Khurshid, S., & Perry, D. E. (2013). Improving bug localization using structured information retrieval. *28th IEEE/ACM international conference on automated software engineering (ASE)* (pp. 345–355). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ASE.2013.6693093>.
- Schröter, A., Bettenburg, N., & Premraj, R. (2010). Do stack traces help developers fix bugs? *7th IEEE working conference on mining software repositories (MSR)* (pp. 118–121). Piscataway, NJ: IEEE. <https://doi.org/10.1109/MSR.2010.5463280>.
- Shokripour, R., Anvik, J., Kasirun, Z. M., & Zamani, S. (2013). Why so complicated? Simple term filtering and weighting for location-based bug report assignment recommendation. *Proceedings of the 10th working conference on mining software repositories* (pp. 2–11). Piscataway, NJ: IEEE. Retrieved from <http://dl.acm.org/citation.cfm?id=2487085.2487089>.
- Sisman, B., & Kak, A. C. (2012). Incorporating version histories in information retrieval based bug localization. *Proceedings of the 9th IEEE working conference on mining software repositories* (pp. 50–59). Piscataway, NJ: IEEE. <https://doi.org/10.1109/MSR.2012.6224299>.
- Sisman, B., & Kak, A. C. (2013). Assisting code search with automatic query reformulation for bug localization. *Proceedings of the 10th working conference on mining software repositories* (pp. 309–318). Piscataway, NJ: IEEE. Retrieved from <http://dl.acm.org/citation.cfm?id=2487085.2487145>.
- Storn, R., & Price, K. (1997). Differential evolution – A simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4), 341–359. <https://doi.org/10.1023/A:10080202821328>.
- Sun, C., Lo, D., Wang, X., Jiang, J., & Khoo, S.-C. (2010). A discriminative model approach for accurate duplicate bug report retrieval. *Proceedings of the 32nd ACM/IEEE international conference on software engineering* (pp. 45–54). New York, NY: ACM. <https://doi.org/10.1145/1806799.1806811>.
- Tian, Y., & Lo, D. (2015). A comparative study on the effectiveness of part-of-speech tagging techniques on bug reports. *22nd international conference on software analysis, evolution, and reengineering (SANER)* (pp. 570–574). Piscataway, NJ: IEEE. <https://doi.org/10.1109/SANER.2015.7081879>.
- Uneno, Y., Mizuno, O., & Choi, E.-H. (2016). Using a distributed representation of words in localizing relevant files for bug reports. *2016 IEEE international conference on software quality, reliability and security (QRS)* (pp. 183–190). Piscataway, NJ: IEEE. <https://doi.org/10.1109/QRS.2016.30>.
- Voorhees, E. M. (2001). The TREC question answering track. *Natural Language Engineering*, 7(4), 361–378. <https://doi.org/10.1017/S1351324901002789>.
- Wang, Q., Parnin, C., & Orso, A. (2015). Evaluating the usefulness of IR-based fault localization techniques. *Proceedings of the 2015 international symposium on software testing and analysis* (pp. 1–11). New York, NY: ACM. <https://doi.org/10.1145/2771783.2771797>.
- Wang, S., & Lo, D. (2014). Version history, similar report, and structure: Putting them together for improved bug localization. *Proceedings of the 22nd international conference on program comprehension* (pp. 53–63). New York, NY: ACM. <https://doi.org/10.1145/2597008.2597148>.
- Wang, S., & Lo, D. (2016). Amlgam ++: Composing rich information sources for accurate bug localization. *Journal of Software: Evolution and Process*, 28(10), 921–942. <https://doi.org/10.1002/smr.1801>.
- Wang, S., Lo, D., & Lawall, J. (2014). Compositional vector space models for improved bug localization. *2014 IEEE international conference on software maintenance and evolution* (pp. 171–180). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ICSME.2014.39>.
- Wang, X., Zhang, L., Xie, T., Anvik, J., & Sun, J. (2008). An approach to detecting duplicate bug reports using natural language and execution information. *Proceedings of the 30th international conference on software engineering* (pp. 461–470). New York, NY: ACM. <https://doi.org/10.1145/1368088.1368151>.
- Wen, M., Wu, R., & Cheung, S.-C. (2016). Locus: Locating bugs from software changes. *31st IEEE/ACM international conference on automated software engineering (ASE)* (pp. 262–273). Piscataway, NJ: IEEE.
- Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80–83. <https://doi.org/10.2307/3001968>.
- Wong, C.-P., Xiong, Y., Zhang, H., Hao, D., Zhang, L., & Mei, H. (2014). Boosting bug-report-oriented fault localization with segmentation and stack-trace analysis. *2014 IEEE international conference on software maintenance and evolution* (pp. 181–190). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ICSME.2014.40>.
- Xia, X., Bao, L., Lo, D., & Li, S. (2016). Automated debugging considered harmful? considered harmful: A user study revisiting the usefulness of spectra-based fault localization techniques with professionals using real bugs from large systems. *2016 IEEE International conference on software maintenance and evolution (ICSME)* (pp. 267–278). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ICSME.2016.67>.
- Ye, X., Bunescu, R., & Liu, C. (2014). Learning to rank relevant files for bug reports using domain knowledge. *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering* (pp. 689–699). New York, NY: ACM. <https://doi.org/10.1145/2635868.2635874>.
- Zhang, H. (2009). An investigation of the relationships between lines of code and defects. *2009 IEEE international conference on software maintenance* (pp. 274–283). Piscataway, NJ: IEEE. <https://doi.org/10.1109/ICSM.2009.5306304>.
- Zhou, J., Zhang, H., & Lo, D. (2012). Where should the bugs be fixed? More accurate information retrieval-based bug localization based on bug reports. *Proceedings of the 34th international conference on software engineering* (pp. 14–24). Piscataway, NJ: IEEE. Retrieved from <http://dl.acm.org/citation.cfm?id=2337223.2337226>.
- Zhou, Y., Tong, Y., Chen, T., & Han, J. (2017). Augmenting bug localization with part-of-speech and invocation. *International Journal of Software Engineering and Knowledge Engineering*, 27(06), 925–949. <https://doi.org/10.1142/S0218194017500346>.